Spring Cloud Gateway

主要内容

Spring Cloud Gateway 简介
使用 Spring Cloud Gateway 实现路由功能
使用 Spring Cloud Gateway 实现过滤器功能
使用 Spring Cloud Gateway 实现熔断功能

学习目标

知识点	要求
Spring Cloud Gateway 简介	了解
使用 Spring Cloud Gateway 实现路由功能	掌握
使用 Spring Cloud Gateway 实现过滤器功能	掌握
使用 Spring Cloud Gateway 实现熔断功能	掌握

Spring Cloud Gateway

一、 Spring Cloud Gateway 简介

1 概述

Spring cloud gateway 是 spring 官方基于 Spring 5.0、Spring Boot2.0 和 Project Reactor 等技术开发的网关，Spring Cloud Gateway 旨在为微服务架构提供简单、有效和统一的 API 路由管理方式，Spring Cloud Gateway 作为 Spring Cloud 生态系统中的网关，目标是替代 Netflix Zuul，其不仅提供统一的路由方式，并且还基于 Filer 链的方式提供了网关基本的功能，例如：安全、监控/埋点、限流等。

2 核心概念

2.1 API 网关

API 网关出现的原因是微服务架构的出现，不同的微服务一般会有不同的网络地址，而外部客户端可能需要调用多个服务的接口才能完成一个业务需求，如果让客户端直接与各个微服务通信，会有以下的问题：

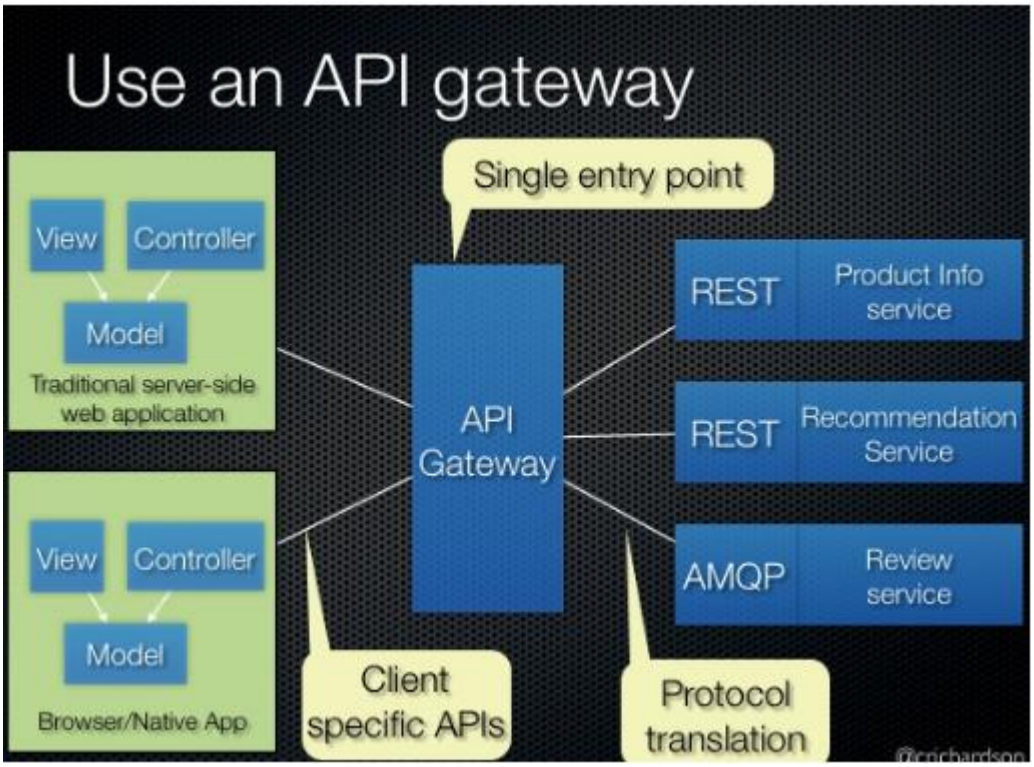
客户端会多次请求不同的微服务，增加了客户端的复杂性。
存在跨域请求，在一定场景下处理相对复杂。
认证复杂，每个服务都需要独立认证。

难以重构，随着项目的迭代，可能需要重新划分微服务。例如，可能将多个服务合并成一个或者将一个服务拆分成多个。如果客户端直接与微服务通信，那么重构将会很难实施。

某些微服务可能使用了防火墙 / 浏览器不友好的协议，直接访问会有一定的困难。

以上这些问题可以借助 API 网关解决。API 网关是介于客户端和服务器端之间的中间层，所有的外部请求都会先经过 API 网关这一层。也就是说，API 的实现方面更多的考虑业务逻辑，而安全、性能、监控可以交由 API 网关来做，这样既提高业务灵活性又不缺安全性。

使用 API 网关后示意图：



使用 API 网关后的优点如下：

易于监控。可以在网关收集监控数据并将其推送到外部系统进行分析。

易于认证。可以在网关上进行认证，然后再将请求转发到后端的微服务，而无须在每个微服务中进行认证。

统一接入。减少了客户端与各个微服务之间的交互次数。

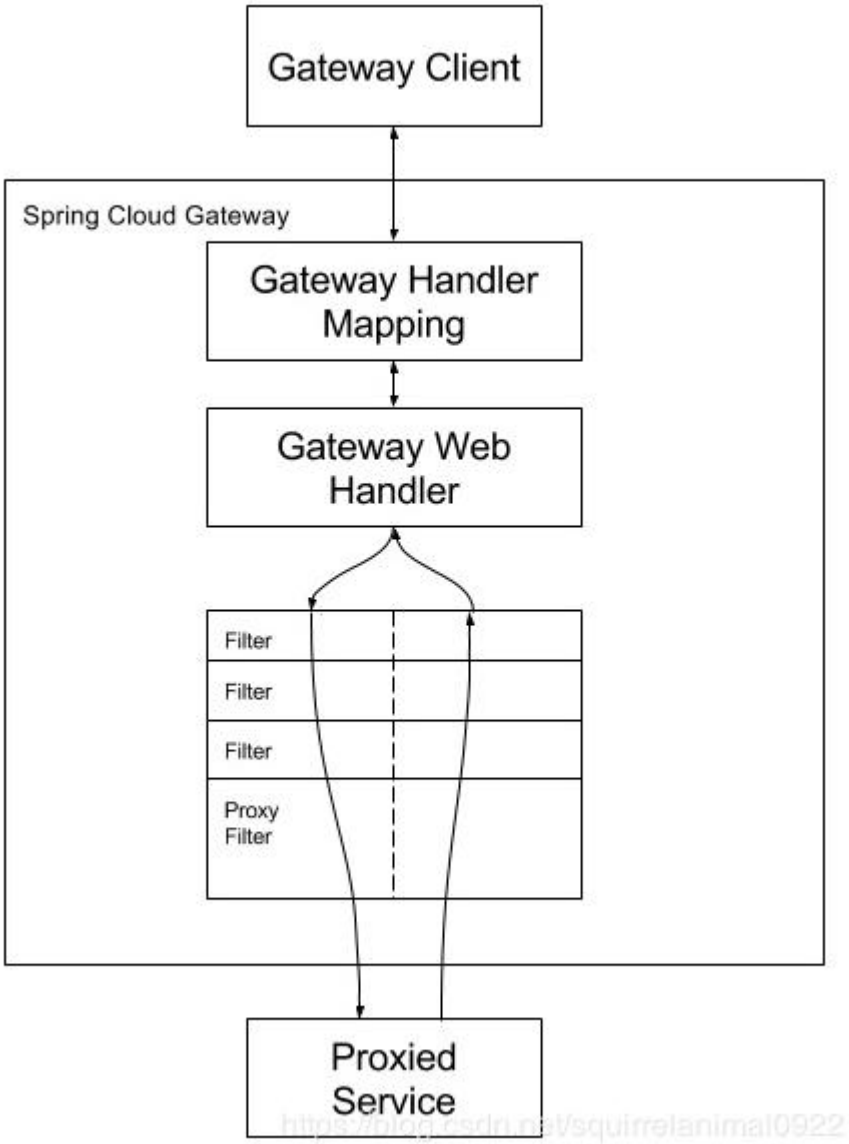
2.2 路由

路由是网关最基础的部分，路由信息有一个 ID、一个目的 URL、一组断言和一组 Filter 组成。如果断言路由为真，则说明请求的 URL 和配置匹配。

2.3 过滤器

一个标准的 Spring webFilter。Spring cloud gateway 中的 filter 分为两种类型的 Filter，分别是 Gateway Filter 和 Global Filter。过滤器 Filter 将会对请求和响应进行修改处理。执

行示意图如下：

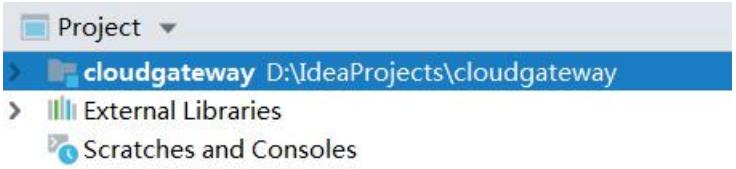


如上图所示，Spring cloudGateway 发出请求。然后再由 Gateway Handler Mapping 中找到与请求相匹配的路由，将其发送到 Gateway web handler。Handler 再通过指定的过滤器链将请求发送到我们实际的服务执行业务逻辑，然后返回。

二、 使用 Spring Cloud Gateway 实现路由功能

1 入门案例

1.1 创建 Gateway 项目



1.1.1 POM 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloudgateway</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
```

```
            <artifactId>spring-cloud-starter-gateway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
    </dependencies>
</project>
```

1.1.2 配置文件

```
server:
  port: 9000
spring:
  application:
    name: cloud-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true  # 开启 Gateway 服务注册和发现的功能
          lower-case-service-id: true  # 将请求路径上的服务名配置为小写
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

1.1.3 启动类

```
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApp {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApp.class, args);
    }
}
```

2 配置文件手工绑定路由规则

2.1 POM 文件

不需要修改。

2.2 配置文件

```yaml
server:
  port: 9000
spring:
  application:
    name: cloud-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: false  # 开启 Gateway 自动服务注册和发现的功能
          lower-case-service-id: true  # 将请求路径上的服务名配置为小写
      routes:
        - id: openfeign-client # 唯一命名
          uri: lb://openfeign-client # 转发路径，lb 代表 loadbalance，即从 Eureka 中获取的服务列表负载均衡器。 openfeign-client 即服务名称，相当于 http://localhost:8082/
          predicates: # 定义映射
            - Path=/client/**  # 发送到 Gateway 网关上的请求路径映射，此映射匹配的路径都会转发到 uri 上。 具体地址为： lb://openfeign-client/client/**
          filters:
            - StripPrefix=1 # 请求转发时，会自动过滤请求转发的第一节地址，即 client。最终地址为： lb://openfeign-client/**
        - id: openfeign-service
          uri: lb://openfeign-service
          predicates:
            - Path=/service/**
          filters:
            - StripPrefix=1

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

2.3 启动类

不需要修改。

三、 使用 Spring Cloud Gateway 实现过滤器功能

Spring Cloud gateway 的 filter 分为两种：GatewayFilter 和 Globalfilter。GlobalFilter 会应用到所有的路由上，而 Gatewayfilter 将应用到单个路由或者一个分组的路由上。

多个过滤器会根据配置文件中的定义来决定执行顺序。

如果提供的自定义过滤器实现了 Ordered 接口，则可以通过接口中的方法 getOrder 来决定执行顺序。具体顺序由 getOrder 方法返回结果升序排列。

1 Spring Cloud Gateway 内置过滤器

GatewayFilter 是一个接口，通过查看源码，其有很多实现类，不同的实现类实现不同的过滤功能。具体如下：

```
Choose Implementation of GatewayFilter (31 found)
1 in AddRequestHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in AddRequestParameterGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in AddResponseHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in DedupeResponseHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in HystrixGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in MapRequestHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in ModifyRequestBodyGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory.rewrite)
1 in PrefixPathGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in PreserveHostHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RedirectToGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RemoveRequestHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RemoveRequestParameterGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RemoveResponseHeaderGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RequestHeaderSizeGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
1 in RequestHeaderToRequestUriGatewayFilterFactory (org.springframework.cloud.gateway.filter.factory)
```

常用过滤器有：

AddRequestParameterGatewayFilterFactory - 在指定请求中增加请求参数的过滤器。

AddRequestHeaderGatewayFilterFactory - 在指定请求中增加请求头参数的过滤器。

StripPrefixGatewayFilterFactory - 在指定请求中处理路径前缀的过滤器。

在 Gateway 中使用过滤器的方式非常简单，只要在配置文件中通过过滤器命名前缀即可快速使用，如使用 AddRequestParameterGatewayFilterFactory，可以通过 AddRequestParameter 在配置文件中配置使用；如使用 StripPrefixGatewayFilterFactory，可以通过 StripPrefix 在配置文件中配置使用。

具体配置过程如下：

```yaml
server:
  port: 9000
spring:
  application:
    name: cloud-gateway
  cloud:
    gateway:
```

```yaml
      discovery:
        locator:
          enabled: false  # 开启 Gateway 自动服务注册和发现的功能
          lower-case-service-id: true  # 将请求路径上的服务名配置为小写
      routes:
        - id: openfeign-client # 唯一命名
          uri: lb://openfeign-client # 转发路径，lb 代表 loadbalance，即从 Eureka 中获取的服务列表负载均衡器。 openfeign-client 即服务名称，相当于 http://localhost:8082/
          predicates: # 定义映射
            - Path=/client/**  # 发送到 Gateway 网关上的请求路径映射，此映射匹配的路径都会转发到 uri 上。 具体地址为： lb://openfeign-client/client/**
          filters:
            - StripPrefix=1 # 请求转发时，会自动过滤请求转发的第一节地址，即 client。最终地址为： lb://openfeign-client/**
            - AddRequestParameter=id, 0 # 在请求中自动增加一个请求参数，参数名为 id，参数值为 0。可以用于提供固定参数。
            - AddRequestParameter=name, testFilter # 每个 AddRequestParameter 过滤器，只能提供一个请求参数，提供多请求参数，需要定义多个过滤器。
            - AddRequestHeader=my-header, test-header-value # 在请求中自动增加一个请求头参数，参数名为 my-header，参数值为 test-header-value。
        - id: openfeign-service
          uri: lb://openfeign-service
          predicates:
            - Path=/service/**
          filters:
            - StripPrefix=1

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

2 自定义过滤器

2.1 自定义 GatewayFilter - 网关过滤器

2.1.1 自定义过滤器

```java
package com.bjsxt.filter;

import org.springframework.cloud.gateway.filter.GatewayFilter;
```

```java
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import
org.springframework.cloud.gateway.filter.factory.AbstractGatewayFilterFactory
;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

import java.util.Arrays;
import java.util.List;

/**
 * 登录校验过滤器工厂
 * 泛型是此工厂构造时自动注入的配置对象。此配置对象由 Spring 容器提供。
 * 根据配置文件中提供的后置配置内容，自动进行配置。
 * 如： 当前类型中定义 Config 有一个属性 token，此属性为字符串类型。那么对应的配置是， Authentication=xxx，xxx 就是要传递的 token 属性值。
 * 在当前实现中，Config 中的 token 属性即是请求参数中用于标明登录标记的请求参数名。
 */
@Component
public class AuthenticationGatewayFilterFactory extends
AbstractGatewayFilterFactory<AuthenticationGatewayFilterFactory.Config> {

    /**
     * 构造方法，此方法调用父类型中的构造，定义构造当前过滤器工厂时，配置参数的具体类型。
     */
    public AuthenticationGatewayFilterFactory() {
        super(AuthenticationGatewayFilterFactory.Config.class);
    }

    /**
     * Spring 构造 Config 对象时，读取的配置文件数据对应 Config 对象中的哪一个属性，即 property 命名。
     * @return
     */
    @Override
    public List<String> shortcutFieldOrder() {
        return Arrays.asList("tokenName");
    }

    /**
```

```
     * 返回过滤器对象。
     * @param config
     * @return
     */
    @Override
    public GatewayFilter apply(Config config) {
        System.out.println("token name is : " + config.getTokenName());
        return new GatewayFilter() {
//ServerWebExchange 命名为服务网络交换器，存放着重要的请求-响应属性、请求实例和响应实例等等
            @Override
            public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
                String token = exchange.getRequest().getQueryParams().getFirst(config.getTokenName());
                System.out.println(exchange.getRequest().getQueryParams());
                if(null == token || token.isEmpty()){
                    // 请求头中不存在 token，用户未登录，返回登录。
                    exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
                    // 直接返回，提示错误。
                    return exchange.getResponse().setComplete();
                }
                // 用户已登录，进入后续执行流程
                return chain.filter(exchange);
            }
        };
    }



    public static class Config {
        private String tokenName;

        public Config() {
        }

        public String getTokenName() {
            return this.tokenName;
        }

        public void setTokenName(String tokenName) {
            this.tokenName = tokenName;
        }
```

```
    }
}
```

2.1.2 配置文件

```yaml
server:
  port: 9000
spring:
  application:
    name: cloud-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: false  # 开启 Gateway 自动服务注册和发现的功能
          lower-case-service-id: true  # 将请求路径上的服务名配置为小写
      routes:
        - id: openfeign-client # 唯一命名
          uri: lb://openfeign-client # 转发路径，lb 代表 loadbalance，即从 Eureka 中获取的服务列表负载均衡器。 openfeign-client 即服务名称，相当于 http://localhost:8082/
          predicates: # 定义映射
            - Path=/client/**  # 发送到 Gateway 网关上的请求路径映射，此映射匹配的路径都会转发到 uri 上。 具体地址为： lb://openfeign-client/client/**
          filters:
            - StripPrefix=1 # 请求转发时，会自动过滤请求转发的第一节地址，即 client。最终地址为： lb://openfeign-client/**
            - AddRequestParameter=id, 0 # 在请求中自动增加一个请求参数，参数名为 id，参数值为 0。可以用于提供固定参数。
            - AddRequestParameter=name, testFilter # 每个 AddRequestParameter 过滤器，只能提供一个请求参数，提供多请求参数，需要定义多个过滤器。
            - AddRequestHeader=my-header, test-header-value # 在请求中自动增加一个请求头参数，参数名为 my-header，参数值为 test-header-value。
            - Authentication=token
        - id: openfeign-service
          uri: lb://openfeign-service
          predicates:
            - Path=/service/**
          filters:
            - StripPrefix=1

eureka:
  client:
```

```
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

2.2 自定义 GlobalFilter - 全局过滤器

GlobalFilter:全局过滤器，不需要在配置文件中配置，作用在所有的路由上，最终通过 GatewayFilterAdapter 包装成 GatewayFilterChain 可识别的过滤器，它为请求业务以及路由的 URI 转换为真实业务服务的请求地址的核心过滤器，不需要配置，系统初始化时加载，并作用在每个路由上。

全局过滤器无需创建工厂类，也无需在配置文件中进行注册。因为其对所有的网关代理的路径都生效。

2.2.1 自定义过滤器

```
package com.bjsxt.filter;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

/**
 * 定义全局日志过滤器
 */
@Component
public class LoggerGlobalFilter implements GlobalFilter {
    private Logger logger = LoggerFactory.getLogger(LoggerGlobalFilter.class);
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
        logger.info("LoggerGlobalFilter run");
        return chain.filter(exchange);
    }
}
```

四、 使用 Spring Cloud Gateway 实现熔断功能

Spring Cloud Gateway 也可以利用 Hystrix 的熔断特性，在流量过大时进行服务降级。

1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloudgateway</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-gateway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
        </dependency>
    </dependencies>
</project>
```

2 熔断处理代码

```java
package com.bjsxt.fallback;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MyFallbackHandler {
    @RequestMapping("/fallback")
    public Object fallback(){
        return "Gateway 熔断处理逻辑";
    }
}
```

3 配置文件

```yaml
server:
  port: 9000
spring:
  application:
    name: cloud-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: false  # 开启 Gateway 自动服务注册和发现的功能
          lower-case-service-id: true  # 将请求路径上的服务名配置为小写
      routes:
        - id: openfeign-client # 唯一命名
          uri: lb://openfeign-client # 转发路径，lb 代表 loadbalance，即从 Eureka 中获取的服务列表负载均衡器。 openfeign-client 即服务名称，相当于 http://localhost:8082/
          predicates: # 定义映射
            - Path=/client/**  # 发送到 Gateway 网关上的请求路径映射，此映射匹配的路径都会转发到 uri 上。 具体地址为： lb://openfeign-client/client/**
          filters:
            - StripPrefix=1 # 请求转发时，会自动过滤请求转发的第一节地址，即 client。最终地址为： lb://openfeign-client/**
            - AddRequestParameter=id, 0 # 在请求中自动增加一个请求参数，参数名为 id，参数值为 0。可以用于提供固定参数。
```

```
            - AddRequestParameter=name, testFilter # 每个AddRequestParameter过滤器，只能提供一个请求参数，提供多请求参数，需要定义多个过滤器。
            - AddRequestHeader=my-header, test-header-value # 在请求中自动增加一个请求头参数，参数名为my-header，参数值为test-header-value。
            - Authentication=token
            - name: Hystrix # 开启Hystrix容错过滤器
              args:
                name: fallbackcmd # 熔断错误
                fallbackUri: forward:/fallback # 出现服务调用问题，访问的本地路径是什么，也就是本地熔断处理逻辑
        - id: openfeign-service
          uri: lb://openfeign-service
          predicates:
            - Path=/service/**
          filters:
            - StripPrefix=1

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

hystrix:
  command:
    default:
      execution:
        timeout:
          # 如果enabled设置为false，则请求超时交给ribbon控制,为true,则超时作为容错根据
          enabled: true
        isolation:
          thread:
            timeoutInMilliseconds: 1000 # 超时时间，默认1000ms
```

4 启动类

```
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;

@SpringBootApplication
```

```java
@EnableCircuitBreaker
public class GatewayApp {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApp.class, args);
    }
}
```

尚学堂·百战程序员